Training for Constraining: speaker notes

Levi Katyal Sledd and Vijay Singh · Devcon 8 workshop

Notes for the final 48-slide deck, `slides/workshop.pdf`

How to read. The notes run continuously in slide order. Each entry shows the slide in its final click state beside the talking track (**Say**), followed by stage directions (**Do**) and backup answers or cautions (**If asked**). *Clicks* counts the advances inside a slide; *start* is minutes into the session. Section-title slides appear as headings.

Core message. A proof system can be perfectly sound while the application is broken, because the proof is about the constraints that were written, not the relation that was meant.

Timing plan (a proposal; calibrate in rehearsal)

Section	Slides	Start	Length	Who
Opening and motivation	1–4	0:00	3:30	
Circuits and constraints	5–11	3:30	7:30	
Patterns, Example 0, Example 1	12–18	11:00	6:00	
Exercise 2 (selector, pairs)	19–21	17:00	5:00	
Example 3 (range, foreign field)	22–26	22:00	4:30	
Deep dive: gnark fake GLV	27–33	26:30	7:00	
Finding underconstraints, demo	34–39	33:30	6:00	
Verification infrastructure, AI	40–43	39:30	3:15	
Close	44	42:45	2:15	
Questions	44	45:00	5:00	

The talk ends at 45:00, leaving five minutes of questions in a 50-minute slot. **If behind schedule,** compress slides 8, 24 and 25 to one sentence each, and protect the exercise (20–21), the fake-GLV arc (28–33), the demo (38) and the close (44).

Before presenting

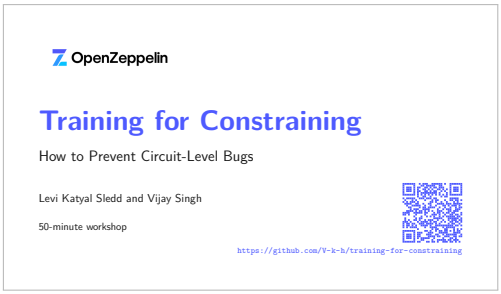
1. **Confirm the slot length.** The title slide says 50 minutes; Devcon 8 lists workshops at 60. With 60, give the extra time to the exercise and to questions.
2. **Slide 4, Airbender row.** A comment in the source questions whether these findings are circuit underconstraints. Verify against the report or drop the row.
3. **Preflight the demo on the presenting laptop**, offline: `make all` and `make lean` from the repository root, with a large terminal font.
4. **Rebuild with** `make slides`. A single `pdflatex` run leaves the title-slide QR code cut off the page; it needs two passes.

Status. The repository is public, and the QR codes on slides 1 and 44 were checked to scan.

Closing line (slide 44): *A prover can replace every private witness and every hint. Only enforced constraints bind the public claim. Check the assumptions of your dependencies and compilers, not only your own code.*

1 Title

start **0:00** clicks **0** pdf page 1



Welcome. One sentence each to introduce yourselves; this is a hands-on session, so keep it short.

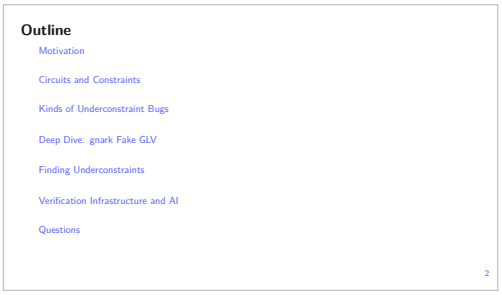
The claim for the session: a SNARK can be perfectly sound and the application still broken, because the proof is about the constraints you wrote, not the relation you meant.

Plan: four small examples you can check by hand, one real bug in a widely used library, and the tools that find these bugs.

Do. Point at the QR code: everything we run today is in this repository and needs only Python 3.10. Pause a few seconds for phones.

2 Outline

start **0:40** clicks **0** pdf page 2



One thread runs through every section: write the intended relation, build an adversarial witness, watch the constraints accept it, repair, then ask how you would have found it.

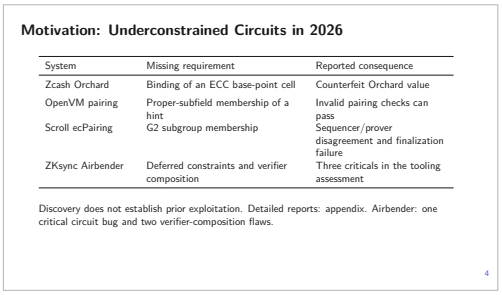
Two interactive moments: a two-minute pair exercise in the middle and a live demo near the end.

Do. Do not read the list aloud.

Slide 3, section title. *Motivation.* One breath, then advance.

4 Motivation: Underconstrained Circuits in 2026

start **1:00** clicks **0** pdf page 4



Read the middle column, not the first. Every row is a property somebody assumed and nobody enforced.

Pick one row and tell it properly. Orchard: an elliptic-curve base-point cell was not bound, and the reported consequence was counterfeit Orchard value. Then name the shape of the others: OpenVM did not check that a hint lay in the right subfield; Scroll did not check G2 subgroup membership.

Mature, audited systems, all this year, the same shape. Then the caveat on the slide: discovery does not establish prior exploitation.

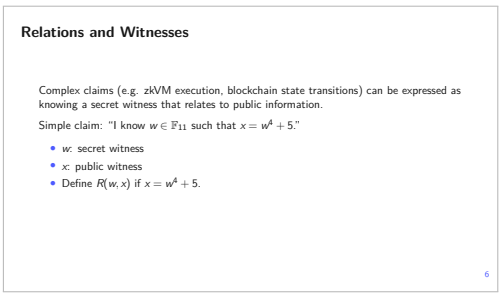
Do. Before the talk: verify or drop the Airbender row (see the cover page).

If asked. Sources for every row are on appendix slide 45. The rows have different consequences; do not say all four were exploitable soundness failures. The slide's own footnote: Airbender was one critical circuit bug and two verifier-composition flaws.

Slide 5, section title. *Circuits and Constraints.* The finite-fields slide is gone, so give the background here in one breath: every value is an element of $\mathbb{F}_p$, and our toy is $\mathbb{F}_{11}$; division exists, since $3 \cdot 4 = 12 \equiv 1$ gives $\frac{1}{3} = 4$; a field has no zero divisors; and field equations wrap around, so they are not integer equations. Slides 18, 21 and 26 rely on exactly these facts.

6 Relations and Witnesses

start **3:30** clicks **2** pdf page 8



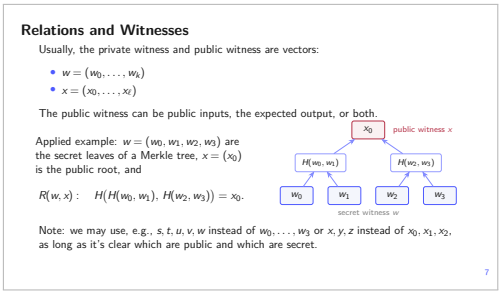
A zero-knowledge statement says: I know something secret that relates to something public.

Simple claim: I know w with $x = w^4 + 5$. Here w is the secret witness, x the public witness, and the relation $R(w, x)$ is the property we actually care about.

Everything else today is one question: does the circuit really enforce R?

7 Relations and Witnesses: Merkle Example

start **4:30** clicks **3** pdf page 12



In practice both witnesses are vectors, and the public part can be inputs, outputs, or both.

Real example: four secret leaves and one public root. The relation says the leaves hash up to the root, $H(H(w_0, w_1), H(w_2, w_3)) = x_0$. Red is public, blue is secret; the deck keeps that colour convention.

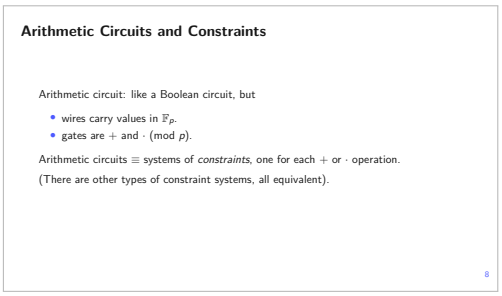
Keep this picture in mind. If a circuit computes a root but never constrains it to equal the public x_0, the proof says nothing about these leaves. That is Example 0.

Do. The tree appears on the third click. Point at the root, then the leaves.

If asked. "A Merkle proof is one leaf plus a path." Yes; the slide shows the whole tree for simplicity. The authentication-path version has the same public and secret split.

8 Arithmetic Circuits and Constraints

start **6:00** clicks **1** pdf page 14



An arithmetic circuit is a Boolean circuit whose wires carry field elements and whose gates are $+$ and $\cdot$ modulo p.

Each gate becomes a polynomial equation, so from here on "circuit" and "constraint system" mean the same thing: a system of polynomial equations, and only those equations are enforced.

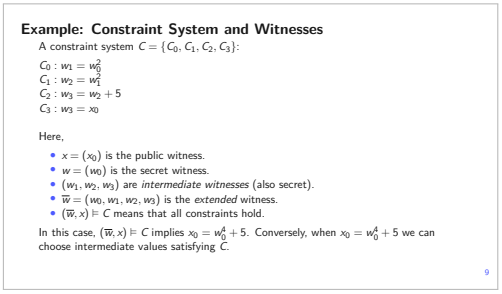
The last line on the slide: other constraint-system formats exist, such as R1CS, PLONKish and AIR, and they are equivalent in what they can express.

Do. If behind schedule, one sentence: every gate is an equation, and only equations are enforced.

If asked. "Equivalent in what sense?" They express the same relations. They differ in cost and in how a bug looks, because one check can be packed into a single gate in one format and spread over several in another.

9 Example: Constraint System and Witnesses

start **6:45** clicks **5** pdf page 20



Four constraints compute $w_0^4 + 5$: square twice, add five. C_3 ties the result to the public x_0.

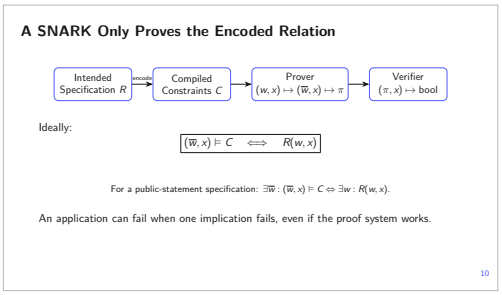
Vocabulary, one click each: x is public; $w = (w_0)$ is secret; w_1, w_2, w_3 are intermediate witnesses, also secret; $\overline{w}$ is the extended witness, everything the prover fills in; $(\overline{w}, x) \vDash C$ means every constraint holds.

Last line: for this system, satisfying C is exactly the relation. Remember C_3. In a few minutes we delete it.

Do. Five clicks, one per bullet.

10 A SNARK Only Proves the Encoded Relation

start **8:30** clicks **2** pdf page 23



The pipeline: an intended specification R is encoded as constraints C; the prover produces a proof; the verifier checks it.

The proof system guarantees one thing: the prover knows an extended witness satisfying C. It knows nothing about R.

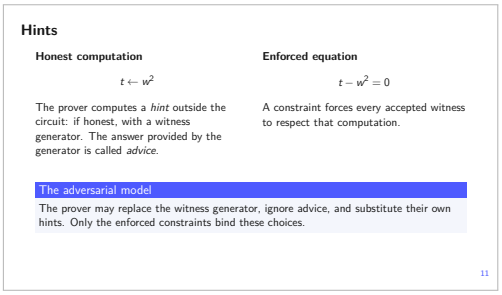
Ideally the two agree in both directions. Left to right is soundness of the circuit: everything accepted is true. Right to left is completeness: every true statement can be proven. Underconstrained means left to right fails, and that is today's topic. The footnote is the weaker, public-statement version with existential quantifiers.

Do. Gesture at the "encode" arrow: every bug today lives on that arrow.

If asked. "What about overconstrained?" That breaks completeness: honest provers get stuck. Annoying, not exploitable, which is why we focus on soundness.

11 Hints

start **9:45** clicks **0** pdf page 24



Some values are expensive or impossible to compute with $+$ and $\cdot$, such as inverses or bit decompositions. The prover computes them outside the circuit. That is a hint; the honest witness generator's answer is called advice.

Left: the honest computation $t \leftarrow w^2$. Right: what is enforced, $t - w^2 = 0$.

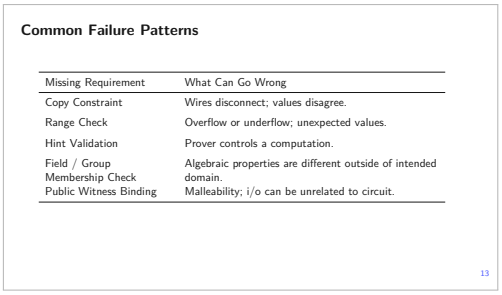
The blue box is the most important sentence today: the prover may replace the witness generator. Anything not constrained is chosen by the attacker.

A hint is not unsafe by itself. It needs enough constraints to establish the property the rest of the circuit relies on, not necessarily a unique value. Slide 18 makes that precise.

Slide 12, section title. *Kinds of Underconstraint Bugs.* Say: four worked examples, each one row of the next table.

13 Common Failure Patterns

start **11:00** clicks **0** pdf page 26



Five missing requirements:

- copy constraints: wires that should agree do not;
- range checks: field elements have no order, so "small" must be enforced;
- hint validation: the prover controls a computation;
- field or group membership: algebraic properties change outside the intended domain, as when cancellation fails off the subgroup;
- public binding: inputs or outputs float free of the circuit.

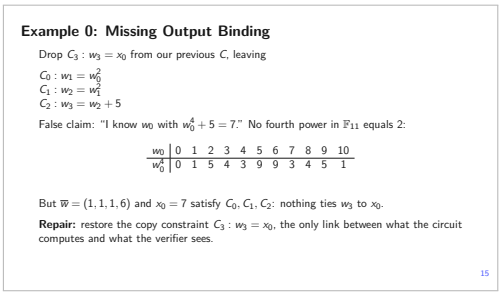
Roadmap: Example 0 is public binding, Example 1 hint validation, Exercise 2 and Example 3 range checks, and fake GLV combines a range problem with group membership. Mention disabled selectors and branch coverage. The categories overlap.

Do. One minute. It is a map, not a lecture.

Slide 14, section title. *Example 0: Missing Public Output Binding.*

15 Example 0: Missing Output Binding

start **12:00** clicks **3** pdf page 31



Start from slide 9 and drop C_3. Ask the room: which constraint mentions x_0 now? None. The private computation still computes $w_0^4 + 5$ into w_3, but nothing connects it to the public value.

Click: a concrete false claim, $w_0^4 + 5 = 7$, so $w_0^4 = 2$. The table lists every fourth power in $\mathbb{F}_{11}$: only $0, 1, 3, 4, 5, 9$. No honest witness exists, so the statement is false.

Click: yet $\overline{w} = (1, 1, 1, 6)$ with $x_0 = 7$ satisfies all three remaining constraints: $1 = 1^2$, $1 = 1^2$, $6 = 1 + 5$. A valid proof of a false statement, with the proof system working perfectly.

Click: the repair restores C_3, the only link between what the circuit computes and what the verifier sees. In the Merkle picture it is the constraint that the computed root equals the public root.

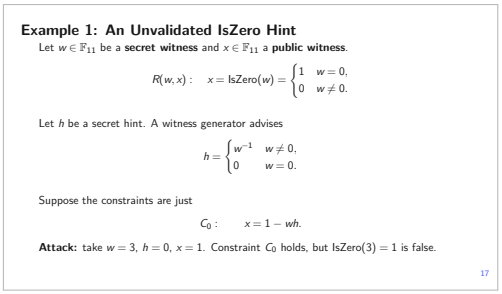
Do. Ask the question on the first view and wait two seconds. This is the "missing public binding" line of `make demo`.

If asked. "Why so few fourth powers?" $\mathbb{F}_{11}^{\times}$ is cyclic of order 10 and $\gcd(4, 10) = 2$, so $w \mapsto w^4$ reaches only five nonzero values.

Slide 16, section title. *Example 1: Unvalidated Hints.*

17 Example 1: An Unvalidated IsZero Hint

start **14:30** clicks **3** pdf page 36



IsZero is everywhere: comparisons, conditionals, "is this balance empty".

Secret w, public output x, relation $x = \mathrm{IsZero}(w)$. IsZero cannot be computed with $+$ and $\cdot$ alone, so a hint helps: $h = w^{-1}$ when $w \neq 0$, otherwise 0. The only constraint is $x = 1 - wh$.

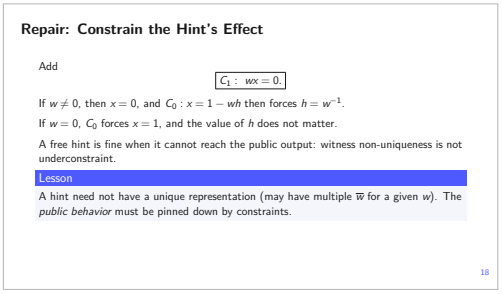
Honest prover: works. Attacker: take $w = 3$ and set $h = 0$. Then $x = 1$, a proof that $\mathrm{IsZero}(3) = 1$. The honest generator never outputs $h = 0$ for nonzero w, but the attacker does not run the honest generator.

If asked. "But w is secret, and $x = 1$ is honest for $w = 0$." True for the gadget in isolation. In a real circuit w is a wire constrained elsewhere, such as a committed balance, and the gadget lets the prover output the wrong bit for that particular w. Soundness is per witness (slide 10).

General attack: for any nonzero w and any target x, set $h = (1 - x)w^{-1}$.

18 Repair: Constrain the Hint's Effect

start **16:00** clicks **1** pdf page 38



Add $wx = 0$.

If $w \neq 0$, then $x = 0$, because a field has no zero divisors; and then $x = 1 - wh$ forces $h = w^{-1}$.

If $w = 0$, the first constraint forces $x = 1$ and h can be anything. That is fine: a free hint is harmless when it cannot reach the public output. Witness non-uniqueness is not underconstraint.

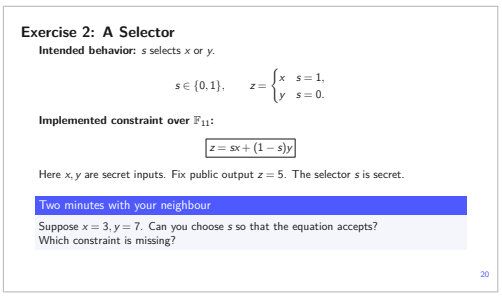
The lesson box: pin down public behaviour, not internal encodings.

If asked. "Isn't a free h a bug?" Over $\mathbb{F}_{11}$ the repaired constraints accept exactly 21 triples (w, h, x), and all 21 have the correct x; h is free only when $w = 0$. Determinism checkers such as Picus ask whether outputs are determined by inputs, and a free h does not violate that.

Slide 19, section title. *Exercise 2: A Selector.* Ask people to find a neighbour.

20 Exercise 2: A Selector

start **17:00** clicks **0** pdf page 40



Intended behaviour: s selects x or y. Implemented: one equation, $z = sx + (1 - s)y$.

Here x, y and s are secret and $z = 5$ is public. With $x = 3$ and $y = 7$: can you find an s that makes the equation accept? Which constraint is missing?

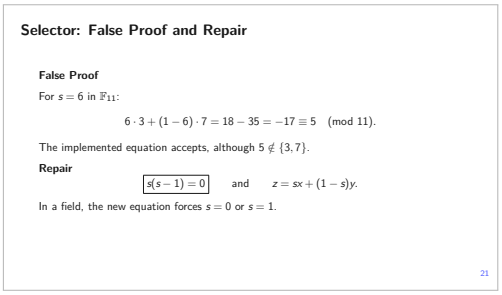
Two minutes with your neighbour.

Do. Start a visible two-minute timer. Do not advance until someone has answered. If the room is stuck, hint: solve the equation for s.

If asked. Answer key: $s = \dfrac{z - y}{x - y} = \dfrac{-2}{-4} = \dfrac{1}{2} = 6$ in $\mathbb{F}_{11}$.

21 Selector: False Proof and Repair

start **20:00** clicks **1** pdf page 42



$s = 6$: $6 \cdot 3 + (1-6) \cdot 7 = 18 - 35 = -17 \equiv 5$, and 5 is neither input.

A nice way to see it: 6 is $\frac{1}{2}$ in $\mathbb{F}_{11}$, and 5 is the midpoint of 3 and 7. The equation is linear interpolation; it accepts every point on the line through x and y.

Repair: $s(s-1) = 0$. No zero divisors, so $s \in \{0, 1\}$ and the output is x or y. The argument uses nothing about 11: it holds in every field, and the repository has it as a machine-checked Lean theorem, `selector_sound`.

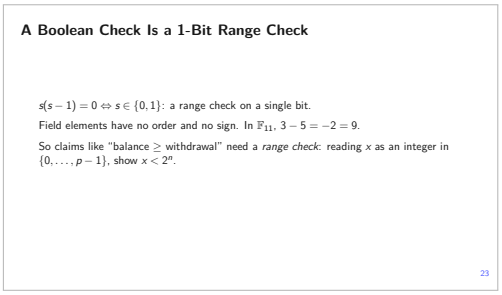
Do. Optional: show the "non-Boolean selector" line of `make demo`, or open `lean/Workshop/Selector.lean`.

If asked. "Why every field?" Only the zero-product rule is used. In $\mathbb{Z}/6$, $s = 3$ satisfies $s(s-1) = 0$ and breaks the selector. For "but x, y are secret", give the IsZero answer (slide 17).

Slide 22, section title. *Example 3: Range Checks and Foreign-Field Arithmetic.* Say: this is the bridge to the gnark case study.

23 A Boolean Check Is a 1-Bit Range Check

start **22:00** clicks **2** pdf page 46

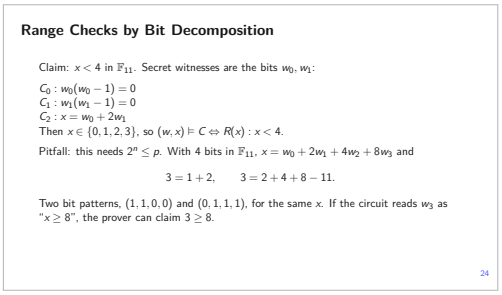


$s(s-1) = 0$ is a range check on a single bit.

Fields have no order and no sign: in $\mathbb{F}_{11}$, $3 - 5 = 9$. So any claim like "balance $\geq$ withdrawal" needs an explicit range check: read the value as an integer in $\{0, \ldots, p-1\}$ and prove it is below 2^n.

24 Range Checks by Bit Decomposition

start **22:45** clicks **2** pdf page 49



To show $x < 4$: two Boolean bits and $x = w_0 + 2w_1$, so $x \in \{0, 1, 2, 3\}$.

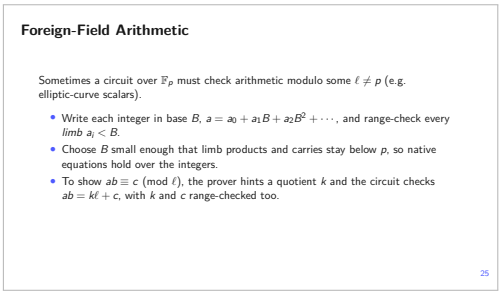
Pitfall: this needs $2^n \leq p$. With four bits in $\mathbb{F}_{11}$, $3 = 1 + 2$ and also $3 = 2 + 4 + 8 - 11$: two bit patterns for the same x. A circuit that reads the top bit as "$x \geq 8$" lets the prover claim $3 \geq 8$.

This is circomlib's Num2Bits pattern. A CAV 2024 crawl found bit-sums in 98% of public Circom projects. It is also the pattern that made SMT solvers time out; we come back to that on slide 39.

Do. If behind schedule, give only the pitfall sentence.

25 Foreign-Field Arithmetic

start **24:00** clicks **3** pdf page 53



Sometimes the circuit lives in $\mathbb{F}_p$ but must check arithmetic modulo a different number ℓ. Elliptic-curve scalars are the standard case, and slide 29 returns to them.

How it is done, one click each:

- write each integer in base B as limbs, and range-check every limb below B;
- choose B small enough that limb products and carries stay below p, so each native equation is also an integer equation, with no wraparound;
- to show $ab \equiv c \pmod{\ell}$, the prover hints a quotient k and the circuit checks $ab = k\ell + c$, with k and c range-checked too.

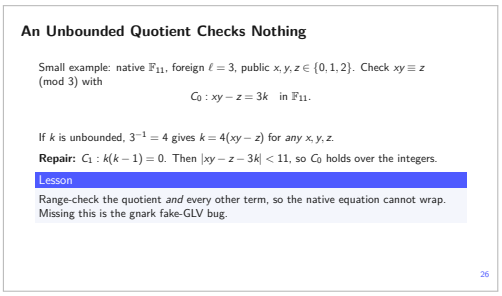
The idea to land: range checks are what let a field equation stand in for an integer equation.

Do. If behind schedule, give only the last bullet and the sentence after it.

If asked. "How small must B be?" Small enough that every product and sum of products, plus carries, stays below p. The exact choice is a cost trade-off.

26 An Unbounded Quotient Checks Nothing

start **25:15** clicks **3** pdf page 57



Toy version: native $\mathbb{F}_{11}$, foreign modulus 3, public $x, y, z \in \{0, 1, 2\}$. We check $xy \equiv z \pmod 3$ with $C_0 : xy - z = 3k$ in $\mathbb{F}_{11}$.

Click: if k is unbounded, $3^{-1} = 4$ gives $k = 4(xy - z)$, which satisfies C_0 for any x, y, z. Say a concrete case, which is not on the slide: the false claim $2 \cdot 2 \equiv 0 \pmod 3$ passes with $k = 16 = 5$, since $3 \cdot 5 = 15 = 4$ in $\mathbb{F}_{11}$. Over the integers $4 = 15$ is false; the field wrapped around.

Click: the repair makes k Boolean. Then $xy - z - 3k$ lies in $\{-5, \ldots, 4\}$, and the only multiple of 11 there is 0, so C_0 holds over the integers and the congruence follows.

Click: the lesson. Range-check the quotient and every other term. "Missing this is the gnark fake-GLV bug" is accurate: in gnark v0.15.0 the quotient was unbounded (slide 31).

Transition: now the same bug inside a widely used library.

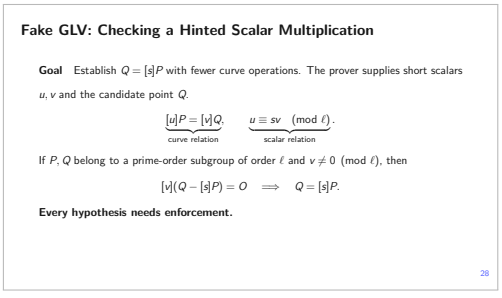
Do. This is the "unbounded foreign-field quotient" line of `make demo`, with $k = 5$.

If asked. "Does a Boolean quotient suffice for real curve arithmetic?" No. It works here only because the numbers are tiny; real gadgets use the limbs of slide 25.

Slide 27, section title. *Deep Dive: gnark Fake GLV.* Say: everything we need is already on the board: two fields, a quotient, and a cancellation.

28 Fake GLV: Checking a Hinted Scalar Multiplication

start **26:30** clicks **0** pdf page 59



Goal: prove $Q = [s]P$ cheaply. Instead of computing $[s]P$ in the circuit, the prover supplies Q and two short scalars u, v. The circuit checks a curve relation $[u]P = [v]Q$ and a scalar relation $u \equiv sv \pmod{\ell}$. Short scalars roughly halve the cost of the curve check.

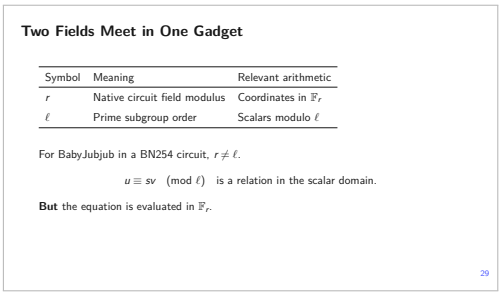
Correctness: substitute to get $[v](Q - [s]P) = O$, then cancel v. Cancelling needs two hypotheses: P, Q in the prime-order subgroup, and $v \not\equiv 0 \pmod{\ell}$.

"Every hypothesis needs enforcement" is the checklist for the next five slides.

If asked. "Why are the scalars short?" Appendix slide 48: lattice reduction outside the circuit gives $|u|, |v| \approx \sqrt{\ell}$.

29 Two Fields Meet in One Gadget

start **27:45** clicks **0** pdf page 60



Two moduli. r is the native field of the circuit, where curve coordinates live. ℓ is the subgroup order, where scalars live.

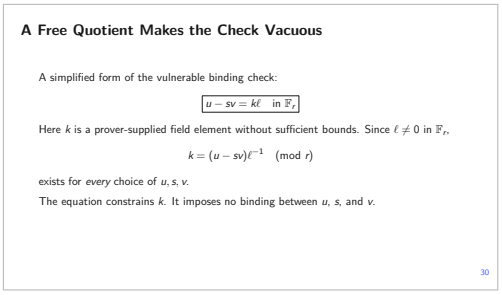
For BabyJubjub inside a BN254 circuit, r is BN254's 254-bit scalar field and BabyJubjub's coordinates are native, but its subgroup order ℓ is a different, 251-bit prime. So $u \equiv sv \pmod{\ell}$ is foreign-field arithmetic: slide 25 again.

The punchline on the slide: *but* the equation is evaluated in $\mathbb{F}_r$.

If asked. Do not conflate coordinate arithmetic, which is native here, with scalar arithmetic, which is foreign. BN254 also has a base field q; that one matters to the verifier, not to the circuit.

30 A Free Quotient Makes the Check Vacuous

start **28:45** clicks **0** pdf page 61



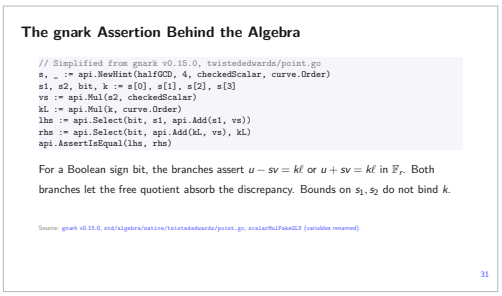
The vulnerable pattern checks $u - sv = k\ell$ in $\mathbb{F}_r$, with k a prover-supplied hint and no bound on it.

Since $\ell \neq 0$ in $\mathbb{F}_r$, $k = (u - sv)\ell^{-1}$ exists for every u, s, v. The equation constrains k and nothing else.

This is slide 26 at 254 bits.

31 The gnark Assertion Behind the Algebra

start **29:45** clicks **0** pdf page 62



Walk the code top to bottom. The hint returns four values: s_1, s_2 (our u, v), a sign bit, and the quotient k. The two Selects implement the two sign branches, and AssertIsEqual checks $u \mp sv = k\ell$ natively.

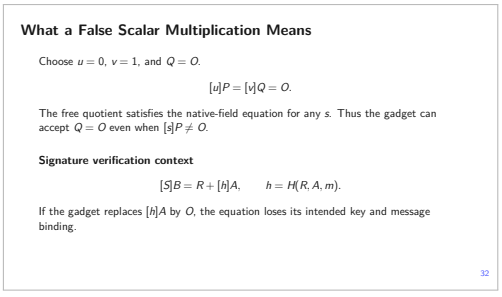
s_1 and s_2 are range-checked to half the bit length. k is never bounded, so in both branches k absorbs any discrepancy.

Do. Checked against gnark's source. The excerpt matches `point.go` in gnark's `std/algebra/native/twistededwards` package at **v0.15.0** (released 13 May 2026), lines 135–147, with variables renamed. In v0.15.0 the public `ScalarMul` for native twisted Edwards curves calls this function directly. The very next line in v0.15.0 already rejects $s_2 = 0$, which is why slide 32 uses $v = 1$ rather than $v = 0$. The source line on the slide links the v0.15.0 file.

If asked. "Which versions? Was it exploited?" State only what the source shows: v0.15.0 has the unbounded quotient; v0.16.0 (19 Aug 2026) replaces it (slide 33). PR #1765, a performance PR merged 15 July 2026, describes wiring scalar-decomposition verification into this path and adds regression tests for malicious decompositions and forged hints. Make no claim about exploitation.

32 What a False Scalar Multiplication Means

start **31:15** clicks **0** pdf page 63



Concrete attack: $u = 0$, $v = 1$, $Q = O$. The curve relation $[0]P = [1]O$ holds trivially, and the free quotient satisfies the scalar relation for any s. So the gadget accepts the identity O as $[s]P$ for every scalar.

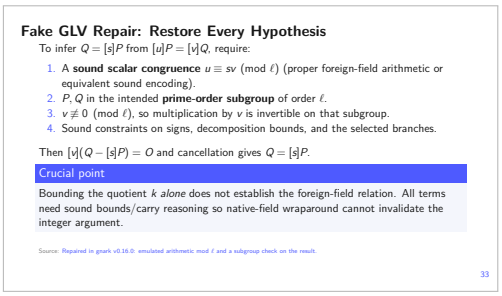
Why that matters: signature verification checks $[S]B = R + [h]A$ with $h = H(R, A, m)$. If $[h]A$ comes from this gadget and the prover sets it to O, the check becomes $[S]B = R$, which anyone satisfies by choosing S and setting $R = [S]B$. The key and the message drop out.

Do. This attack follows from reading the v0.15.0 code on slide 31. It was derived from the source, not run against gnark.

If asked. "So every user was forgeable?" No such claim. Reachability depends on how an application called the gadget and which version it pinned.

33 Fake GLV Repair: Restore Every Hypothesis

start **32:15** clicks **0** pdf page 64



The checklist from slide 28:

1. a sound scalar congruence, real foreign-field arithmetic rather than a free quotient;
2. P, Q in the prime-order subgroup;
3. $v \not\equiv 0 \pmod{\ell}$;
4. bounds on signs, decomposition and branches.

With all four, $[v](Q - [s]P) = O$ and cancellation gives $Q = [s]P$.

The crucial-point box: bounding k alone is not enough. Every term needs bounds, exactly as on slide 26.

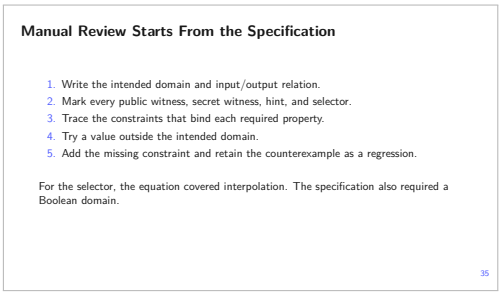
Do. **gnark v0.16.0 implements this list** (checked in its source). The decomposition is verified with emulated arithmetic modulo the curve order, so the native quotient is gone (item 1). $s_2 \neq 0$ is asserted in that emulated field (item 3). The hinted point is forced into the prime-order subgroup, with a source comment describing a 2-torsion attack otherwise (item 2). s_1, s_2 are range-checked (item 4).

If asked. "Is the cancellation step proved?" Yes, in the model: the repository's Lean theorem `scalar_cancellation` proves it over any vector space, from the scalar relation and $v \neq 0$. Item 2 is what makes the prime-order group a vector space over $\mathbb{F}_\ell$.

Slide 34, section title. *Finding Underconstraints.* Say: we have seen five bugs. How would you have found them?

35 Manual Review Starts From the Specification

start **33:30** clicks **0** pdf page 66



Five steps:

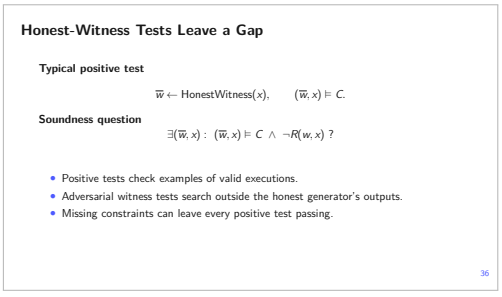
1. write the intended domain and relation before reading constraints;
2. mark every value as public, secret, hint or selector;
3. for each property the specification needs, find the constraint that enforces it;
4. try a value outside the intended domain;
5. add the missing constraint and keep the counterexample as a regression test.

The intended domain is the set of values each variable is supposed to take, such as $s \in \{0, 1\}$ or a small k, as opposed to all of $\mathbb{F}_p$. For the selector, the equation covered the interpolation, but the specification also required s Boolean, and nothing enforced that.

If asked. Reviewers must also read call sites and dependency assumptions. Fake GLV was a library bug, invisible from application code. Step 4 is done by hand here; slides 37–39 automate it.

36 Honest-Witness Tests Leave a Gap

start **34:30** clicks **0** pdf page 67



A typical test generates the witness honestly and checks that the constraints hold. That tests completeness.

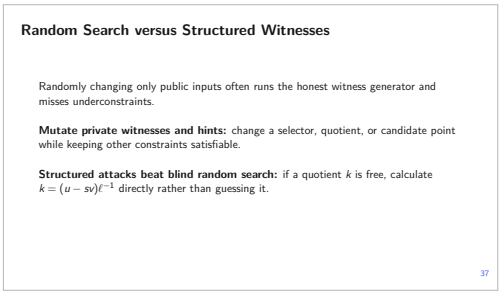
The soundness question is different: does any extended witness, honest or not, satisfy C while its real witness violates R?

Every example today passes every honest test, because the honest generator never produces $s = 6$ or $h = 0$.

If asked. Do not say testing is useless: negative tests with injected witnesses do catch these bugs.

37 Random Search versus Structured Witnesses

start **35:15** clicks **0** pdf page 68



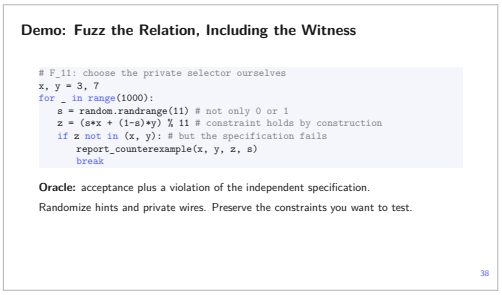
Fuzzing public inputs just reruns the honest generator. Fuzz the private witness and the hints instead.

Sometimes random works: for the selector, 9 of the 11 values of s in $\mathbb{F}_{11}$ are counterexamples, and over a large field almost all of them are.

Sometimes it cannot: a random k in a 254-bit field essentially never equals $(u-sv)\ell^{-1}$, but you can compute that value directly. Structure beats luck.

38 Demo: Fuzz the Relation, Including the Witness

start **36:00** clicks **0** pdf page 69



The oracle is the independent specification: the constraints accept, and $z \notin \{x, y\}$.

Mutate private wires and hints; keep the constraints you are testing.

Then switch to the terminal and show it.

Do. From the repository root run `make demo`. Point at the non-Boolean selector entry: witness $(3, 7, 5; 6)$, vulnerable ACCEPT, repaired REJECT. Optionally `make test`: seven tests, exhaustive over $\mathbb{F}_{11}$.

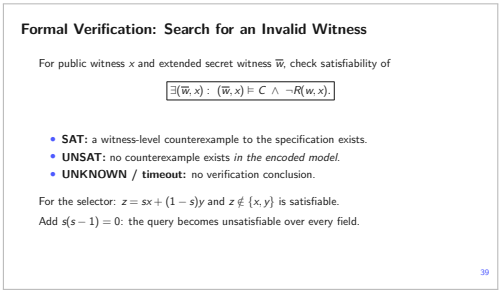
The loop on the slide is a sketch of what `make demo` checks; it is not a file in the repository. It computes z from the constraint, so the constraint always holds, and the only test left is the specification.

Fallback without a terminal: slide 21 already shows the result.

If asked. The demo prints x, y, z as public, while the slides make x, y secret. The arithmetic is identical.

39 Formal Verification: Search for an Invalid Witness

start **38:00** clicks **1** pdf page 71



Soundness is a "for all": every accepted witness satisfies R. A solver proves it by refuting the negation: is there an extended witness that satisfies C while its real witness violates R?

- SAT returns a concrete exploit;
- UNSAT means no counterexample exists in the encoded model;
- UNKNOWN or timeout means nothing.

We ran the selector in cvc5's finite-field theory. Without booleanity: SAT, and pinned to $(3, 7, 5)$ it finds $s = 6$ in milliseconds. With $s(s-1) = 0$: UNSAT, also over the 254-bit BN254 field.

This is not fuzzing. UNSAT comes from algebra: a certificate that 1 lies in the ideal generated by the constraints together with the encoded $\neg R$. The test runs one way: 1 in the ideal proves soundness, but 1 not in the ideal does not by itself mean a bug, so the solver then searches for a point over $\mathbb{F}_p$.

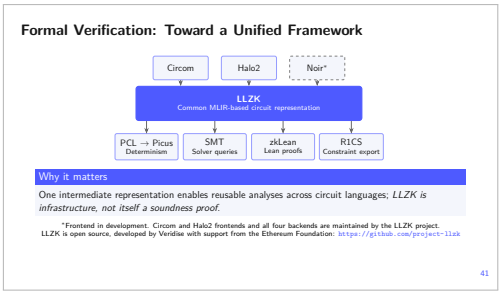
Do. The companion note `notes/smt-ring-theory.pdf` in the repository has the encoding of $\neg R$, the certificate, and the split Gröbner basis story.

If asked. "Why not always use SMT?" Gröbner-basis reasoning is worst-case doubly exponential. Bit decompositions (slide 24) were the classic blow-up; split Gröbner bases (CAV 2024) raised solved benchmarks from 475 to 969 of 1140, but integer division and remainder, the lifting argument of slide 26, remain hard. That is where Lean proofs take over.

Slide 40, section title. *Verification Infrastructure and AI.*

41 Formal Verification: Toward a Unified Framework

start **39:30** clicks **0** pdf page 73



Problem: every circuit language would need its own analysers.

LLZK is a common MLIR-based representation. Circom and Halo2 compile into it today, and a Noir frontend is in development: the dashed box. Four backends run on it: PCL, which feeds the Picus determinism checker; SMT for solver queries like slide 39's; zkLean for Lean proofs; and R1CS export. That is exactly the set the LLZK project lists as maintained.

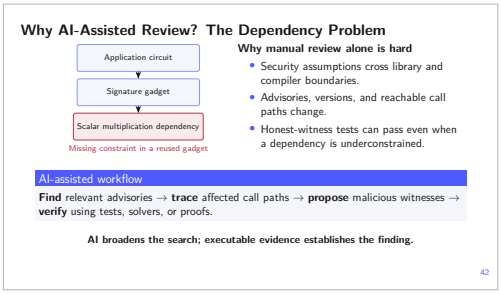
The box says it plainly: LLZK is infrastructure, not itself a soundness proof.

Credit: open source, developed by Veridise with support from the Ethereum Foundation.

If asked. "What about Zirgen?" RISC Zero's circuit compiler. Veridise has announced a zirgen-to-LLZK frontend, but it is not on LLZK's maintained list. Zirgen writes constraints; solvers decide them. "Plonky3?" It appears in LLZK's benchmarks, not as a maintained frontend, so it is not on the slide.

42 Why AI-Assisted Review? The Dependency Problem

start **40:45** clicks **0** pdf page 74



The fake-GLV story as a picture: the application circuit is fine, the signature gadget is fine, and the bug sits two layers down in a reused scalar-multiplication dependency.

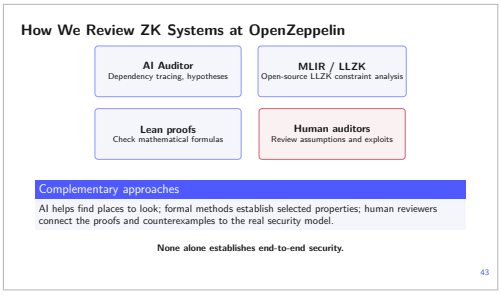
Manual review alone struggles: assumptions cross library and compiler boundaries, and advisories, versions and call paths keep changing.

AI helps find where to look: advisories, call paths, candidate malicious witnesses. The last step is non-negotiable: verify with tests, solvers or proofs. Read the bottom line as written.

If asked. Present AI as triage, not as proof of an exploit and not as a substitute for human analysis.

43 How We Review ZK Systems at OpenZeppelin

start **41:45** clicks **0** pdf page 75



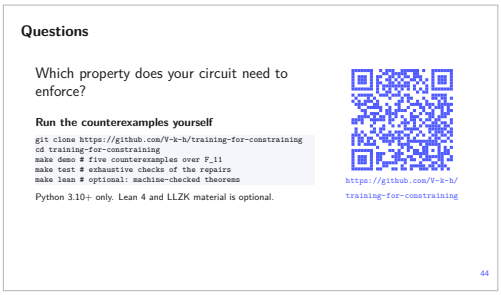
Four complementary approaches: an AI auditor for dependency tracing and hypotheses; the open-source LLZK framework from slide 41, built by Veridise, to analyse circuit constraints; Lean to check the mathematics; human auditors to connect proofs and counterexamples to the real threat model.

None alone establishes end-to-end security.

If asked. Careful wording: not every engagement uses every technique.

44 Questions

start **42:45** clicks **0** pdf page 76



Ask the question on the slide: which property does your circuit need to enforce?

Then the takeaway: a prover can replace every private witness and every hint. Only enforced constraints bind the public claim. Check the assumptions of your dependencies and compilers, not only your own code.

Everything today runs from this repository with Python alone. Questions run from 45:00.

Do. Leave this slide up through questions so people can scan the QR code.

If asked. **Must a hint have a unique value?** No (slide 18). **Does an on-curve check imply subgroup membership?** Not in general; gnark v0.16.0 added an explicit subgroup check for that reason. **Why is a SAT witness better than a warning?** It is an executable counterexample that becomes a regression test.

45 Appendix: Source Reports

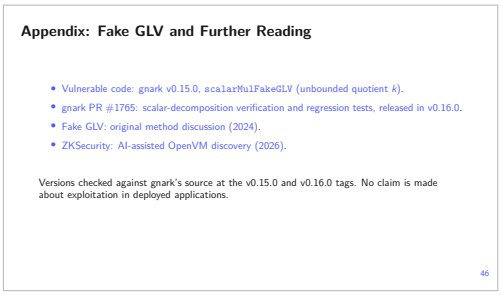
start **backup** clicks **0** pdf page 77



Use only if asked about the incidents on slide 4. Every link on this slide resolved when checked on 28 October 2026.

46 Appendix: Fake GLV and Further Reading

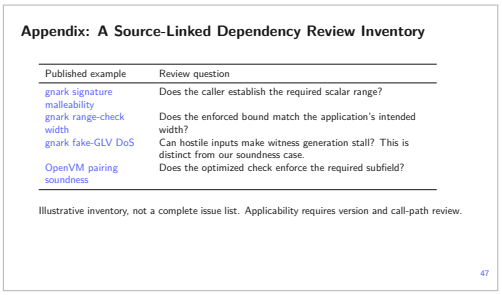
start **backup** clicks **0** pdf page 78



Links to the vulnerable gnark v0.15.0 code, PR #1765 (released in v0.16.0), and the original ethresear.ch write-up of the method.

47 Appendix: A Source-Linked Dependency Review Inventory

start **backup** clicks **0** pdf page 79

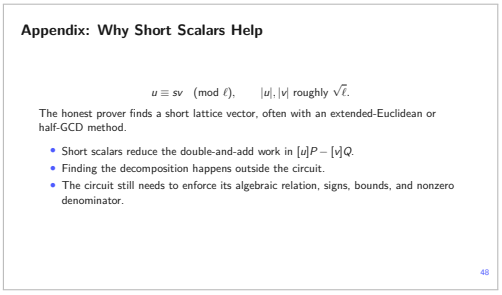


Example review questions per published advisory. Use it if someone asks how to start a dependency review.

If asked. The fake-GLV DoS advisory listed here is a different issue from the soundness case in the main deck.

48 Appendix: Why Short Scalars Help

start **backup** clicks **0** pdf page 80



For "why short scalars". Lattice reduction outside the circuit finds $u \equiv sv \pmod{\ell}$ with $|u|, |v| \approx \sqrt{\ell}$, which halves the double-and-add loop. Because the decomposition happens outside the circuit, the circuit must enforce the relation, signs, bounds and a nonzero v.

If asked. gnark's v0.15.0 hint used a half-GCD lattice decomposition; v0.16.0 uses LLL-based rational reconstruction.